

Aula 02

O JavaScript que profissionais escrevem

`map` · `filter` · `reduce` · desestruturação · spread

Combinado de hoje

Vamos ver **muitos** métodos novos.

Você não precisa decorar nenhum

Precisa saber que **existem** e onde procurar.

Tudo está em `recursos/METODOS-NATIVOS.md`

Programador consulta documentação o dia inteiro.
Isso não é fraqueza — é o trabalho.

Hoje é a aula mais importante das 3 primeiras

Tudo que vem depois — Express, banco de dados, React —
é escrito com o que aprendemos hoje.

Se hoje ficar bem entendido, as próximas 8 aulas ficam fáceis.

const, let e o var aposentado

```
const nome = "Maria";    // não troco o valor  
let contador = 0;        // vou trocar  
var coisa = 1;           // ✗ não use mais
```

Regra do curso

Comece sempre com `const` .

Só mude para `let` quando o JS reclamar.

! A pegadinha do `const`

```
const lista = [1, 2, 3];  
  
lista.push(4);    // ✅ funciona!  
lista = [9, 9, 9]; // ❌ TypeError
```

`const` **não** é "conteúdo imutável".

É "esta variável aponta sempre para **esta caixa**".

Você não troca a caixa. Mas mexe no que tem dentro.

Arrow functions

```
// clássica
function dobrar(n) { return n * 2; }

// arrow com corpo
const dobrar = (n) => { return n * 2; };

// arrow curta – return implícito ★
const dobrar = (n) => n * 2;
```

⚠ Retornando objeto de uma arrow

```
const criar = (nome) => { nome: nome };    // ✗ undefined  
  
const criar = (nome) => ({ nome: nome });  // ✓  
//                                     ↑      ↑  
//                               parênteses salvam
```

Sem parênteses, `{}` é o **corpo** da função, não um objeto.

Nossos dados de hoje

```
const vendas = [  
  { produto: "Notebook", categoria: "eletronicos",  
    valor: 3500, quantidade: 2 },  
  { produto: "Mouse", categoria: "eletronicos",  
    valor: 80, quantidade: 10 },  
  { produto: "Cadeira", categoria: "moveis",  
    valor: 900, quantidade: 3 },  
  // ...  
];
```

Antes: como ler um método

```
vendas.map( (venda) => venda.produto )
```

↑
↑
↑
a FUNÇÃO que você entrega
o NOME do método
em QUEM você chama

3 perguntas para todo método novo

1. O que ele devolve?
2. Ele altera o original?
3. O que acontece se não achar nada?

O trio essencial

Método	Pergunta que ele responde	Entram → saem
map	"vire outra coisa"	5 → 5
filter	"você fica ou sai?"	5 → 0..5
reduce	"vá acumulando"	5 → 1



.map()

`.map()` — a ficha técnica

O que faz: transforma cada item da lista

Quando usar: "quero a mesma lista,
mas com cada item virando outra coisa"

Devolve	um array novo , do mesmo tamanho
Altera o original?	× não
Se a lista for vazia?	devolve <code>[]</code>

`.map()` na prática

```
[1, 2, 3].map((n) => n * 2);  
// [2, 4, 6]  
  
vendas.map((v) => v.produto);  
// ["Notebook", "Mouse", "Cadeira", ...]  
  
vendas.map((v) => ({  
  produto: v.produto,  
  total: v.valor * v.quantidade,  
}));
```

Entram 5, saem 5. Sempre.

⚠ Pegadinha do `.map()`

Devolvendo um **objeto** numa arrow curta:

```
lista.map((x) => { nome: x })           // ✗ undefined
lista.map((x) => ({ nome: x })))        // ✓
//                                     ↑      ↑
//                               parênteses salvam
```

Sem os parênteses, o JS acha que `{}` é o **corpo** da função.



.filter()

`.filter()` — a ficha técnica

O que faz: mantém só os itens aprovados

Quando usar: "quero **só alguns** itens desta lista"

Devolve	array novo , com 0 a N itens
Altera o original?	× não
Se nada bater?	devolve <code>[]</code> — nunca <code>undefined</code>

`.filter()` na prática

```
[1, 2, 3, 4].filter((n) => n % 2 === 0);  
// [2, 4]  
  
vendas.filter((v) => v.categoria === "eletronicos");  
  
alunos.filter((a) => a.ativo);
```

A função dentro responde **sim ou não**.

`true` o item fica · `false` o item sai.

Truque útil do `.filter()`

Remover um item **sem alterar** o array:

```
// por valor  
lista.filter((item) => item.id !== 3)  
  
// por posição  
lista.filter((item, indice) => indice !== 2)
```

Você vai usar isso muito no React. ★



.reduce()

o que assusta todo mundo

.reduce() — a ficha técnica

O que faz: percorre a lista **acumulando**
e devolve **um único valor**

Quando usar: somar, contar, agrupar,
achar o maior — qualquer coisa que **resuma**

Devolve	um valor só (número, objeto, array...)
Altera o original?	× não
Parâmetros	a função e o valor inicial

reduce — resumir

```
const faturamento = vendas.reduce((acc, v) => {  
  return acc + v.valor * v.quantidade;  
}, 0);  
//   ↑ valor inicial: NUNCA esqueça
```

Repare: a função recebe **dois** parâmetros.

`acc` é o que veio acumulado até agora.

reduce rodada a rodada

#	acc chega	item	devolve
1	0	Notebook	7000
2	7000	Mouse	7800
3	7800	Cadeira	10500
4	10500	Mesa	11700
5	11700	Teclado	12950

O que sai numa rodada é o **acc** da próxima.

Encadeando: o código vira frase

```
const total = vendas
  .filter((v) => v.categoria === "eletronicos")
  .map((v) => v.valor * v.quantidade)
  .reduce((soma, valor) => soma + valor, 0);
```

"Das vendas, **fique só** com eletrônicos,
transforme no total de cada, e **some** tudo."

O mesmo com **for**

```
let total = 0;
for (let i = 0; i < vendas.length; i++) {
  if (vendas[i].categoria === "eletronicos") {
    total += vendas[i].valor * vendas[i].quantidade;
  }
}
```

Funciona igual.

Mas diz **como fazer**, não o que quero.

O acumulador não precisa ser número

Pode ser um **objeto** — é assim que se agrupa:

```
const porCategoria = vendas.reduce((acc, v) => {  
  return { ...acc, [v.categoria]: (acc[v.categoria] ?? 0) + 1 };  
}, {});  
//      ↑ começa com objeto vazio  
  
// { eletronicos: 3, moveis: 2 }
```

Os outros do dia a dia

`.find()` — o primeiro que bater

```
vendas.find((v) => v.produto === "Mouse");  
// { produto: "Mouse", valor: 80, ... }  
  
vendas.find((v) => v.produto === "Monitor");  
// undefined
```

| **Devolve** | o objeto, ou `undefined` |

| **Quando usar** | quando espera **um** resultado |

! **find** × **filter**

	devolve	se não achar
<code>.find()</code>	o objeto	<code>undefined</code>
<code>.filter()</code>	um array	<code>[]</code> (vazio)

```
vendas.find((v) => v.id === 99)    // undefined
vendas.filter((v) => v.id === 99)  // []
```

Espera **um** → `find`

Espera **vários** → `filter`

`.findIndex()` — a posição

```
vendas.findIndex((v) => v.id === 3);    // 2  
vendas.findIndex((v) => v.id === 99);   // -1
```

⚠ Sempre teste com `=== -1`

```
if (indice === -1) { /* não achou */ } // ✓  
if (!indice) { /* BUG! */ }           // ✗
```

A posição `0` também é "falsa" em JavaScript.

`.some()` e `.every()`

```
vendas.some((v) => v.valor > 3000);    // true  
vendas.every((v) => v.valor > 50);    // true
```

| `.some()` | "existe **pelo menos um**?" |

| `.every()` | "**todos** atendem?" |

Devolvem `true` ou `false`.

Uso real: verificar duplicidade antes de criar

→ `filmes.some(f => f.titulo === novo)`

`.includes()` — está na lista?

```
["baixa", "media", "alta"].includes("alta")    // true
["baixa", "media", "alta"].includes("urgente")  // false

"Estudar Node".includes("Node")                // funciona em texto também
```

| **Devolve** | `true` / `false` |

| **Quando usar** | validar valores permitidos ★ |

Diferença para `.some()`: `includes` compara **valores**,
`some` recebe uma **função**.

`.sort()` — o traiçoeiro

```
[...vendas].sort((a, b) => b.valor - a.valor);  
//  ↑ copie ANTES
```

`.sort()` ALTERA o array original

```
lista.toSorted((a, b) => a - b) // versão moderna, não altera  
[...lista].sort((a, b) => a - b) // clássica: copia antes
```

Como funciona a comparação

A função recebe dois itens e devolve um **número**:

Devolveu	Significa
negativo	a vem antes
zero	tanto faz
positivo	b vem antes

```
(a, b) => a - b           // crescente
(a, b) => b - a           // decrescente
(a, b) => a.localeCompare(b) // texto em português
```

⚠️ `sort()` sem função = desastre

```
[10, 9, 100].sort()  
// [10, 100, 9] 🤯
```

Sem função, ele compara como **TEXTO**.

E "100" vem antes de "9" no alfabeto.

```
[10, 9, 100].sort((a, b) => a - b)  
// [9, 10, 100] ✅
```

`.localeCompare()` — acentos

```
["Ávila", "Bruno", "Ana"].sort()  
// ordem errada: acentos vão para o fim  
  
["Ávila", "Bruno", "Ana"]  
  .sort((a, b) => a.localeCompare(b))  
// ["Ana", "Ávila", "Bruno"] ✓
```

Para ordenar **nomes em português**,
sempre `localeCompare`.

`.join()` — array vira texto

```
["a", "b", "c"].join(", ")    // "a, b, c"  
["a", "b", "c"].join("")      // "abc"  
erros.join(" · ")             // "erro 1 · erro 2"
```

É o oposto do `.split()`:

```
"a,b,c".split(",")           // ["a", "b", "c"]
```

`.forEach()` — só executar

```
lista.forEach((item) => console.log(item));
```

Devolve `undefined`. Não monta lista nova.

Quer uma lista nova? → `.map()`

Só quer fazer algo? → `.forEach()`

⚠ Não funciona com `await` (aula 03)

Desestruturação

```
const usuario = { nome: "Maria", idade: 28 };

// antes
const nome = usuario.nome;

// agora
const { nome, idade } = usuario;

// com renomeação e padrão
const { nome: nomeCompleto, pais = "Brasil" } = usuario;
```

Desestruturação em parâmetros

```
// em vez de
function saudar(usuario) {
  return `Olá, ${usuario.nome}`;
}

// escreva
function saudar({ nome }) {
  return `Olá, ${nome}`;
}
```

Você verá isso em **todo** código de Express e React.

Spread ... — espalhar

```
const numeros = [1, 2, 3];  
const mais = [...numeros, 4, 5];    // [1,2,3,4,5]  
  
const usuario = { nome: "Maria", idade: 28 };  
const novo = { ...usuario, idade: 29 };
```

Copiar e mudar uma coisa.

O padrão mais importante da aula. ★

Rest `...` — juntar

```
function somarTodos(...numeros) {  
  return numeros.reduce((t, n) => t + n, 0);  
}  
somarTodos(1, 2, 3, 4);    // 10  
  
const { nome, ...resto } = usuario;
```

Mesmo símbolo, papel oposto — depende de onde está.

Imutabilidade: não mexa no original

```
// ❌ muta o array de quem chamou  
function adicionar(lista, item) {  
  lista.push(item);  
  return lista;  
}
```

```
// ✅ devolve um array novo  
function adicionar(lista, item) {  
  return [...lista, item];  
}
```

Por que isso importa MESMO

1. Ninguém é pego de surpresa por um dado que mudou sozinho
2. O React só redesenha a tela quando vê um objeto novo

Se você mutar, a tela **não atualiza**.

É o erro nº 1 de quem começa em React.

(estamos plantando isso 6 aulas antes)

Cola da imutabilidade

 muta	 não muta
<code>lista.push(x)</code>	<code>[...lista, x]</code>
<code>lista.unshift(x)</code>	<code>[x, ...lista]</code>
<code>lista.splice(i,1)</code>	<code>lista.filter((_ ,j) => j !== i)</code>
<code>obj.a = 1</code>	<code>{ ...obj, a: 1 }</code>
<code>lista.sort()</code>	<code>[...lista].sort()</code>

Percorrendo objetos

Objetos não têm `.map()`

```
const contagem = { drama: 2, acao: 1 };  
contagem.map(...) // ✗ não existe
```

`Object.entries()` destrava isso

```
Object.entries(contagem)  
// [["drama", 2], ["acao", 1]] ← virou array!
```

E arrays têm `map`, `filter`, `sort` ...

A família **Object**

```
const usuario = { nome: "Ana", idade: 28 };

Object.keys(usuario)      // ["nome", "idade"]
Object.values(usuario)    // ["Ana", 28]
Object.entries(usuario)   // [["nome", "Ana"], ["idade", 28]]
```

Todas devolvem **array**.

Uso prático: `Object.keys(obj).length === 0`

→ "esse objeto está vazio?"

Percorrendo com desestruturação

```
for (const [categoria, total] of Object.entries(dados)) {  
  console.log(`${categoria}: ${total}`);  
}  
//           ↑           ↑  
//      as duas posições do par [chave, valor]
```

```
Object.entries(dados)  
  .map(([nome, total]) => ({ nome, total }))  
  .sort((a, b) => b.total - a.total);
```

Formatando números

.toFixed(casas)

```
(3.14159).toFixed(2)    // "3.14"  
(10 / 3).toFixed(1)    // "3.3"  
(8).toFixed(2)         // "8.00"
```

⚠ Devolve TEXTO, não número!

```
(3.14159).toFixed(2) + 1    // "3.141" 🧑  
Number((3.14159).toFixed(2)) // 3.14 ✅
```

Use para **mostrar**, não para calcular.

`.toLocaleString("pt-BR")`

```
(1234.5).toLocaleString("pt-BR")  
// "1.234,5"  
  
(1234.5).toLocaleString("pt-BR", {  
  style: "currency",  
  currency: "BRL"  
})  
// "R$ 1.234,50"
```

Ponto no milhar, vírgula no decimal —
do jeito brasileiro, sem você fazer conta.

`.padEnd()` — alinhando no terminal

```
"Ana".padEnd(12)      // "Ana           "  
"Bernardo".padEnd(12) // "Bernardo        "  
  
console.log("Ana".padEnd(12) + "R$ 100");  
console.log("Bernardo".padEnd(12) + "R$ 250");
```

```
Ana           R$ 100  
Bernardo      R$ 250
```

`padStart()` faz o mesmo, mas à esquerda.

```
"5".padStart(2, "0") → "05"
```

A família **Math**

Função	<code>Math.x(4.3)</code>	<code>Math.x(4.7)</code>
<code>Math.round</code>	4	5
<code>Math.ceil</code> (sempre ↑)	5	5
<code>Math.floor</code> (sempre ↓)	4	4

```
Math.max(3, 7, 2)           // 7
Math.max(...[3, 7, 2])      // 7 ← array precisa de spread
Math.max(...lista.map(i => i.id)) // maior id ★
```

Três extras que salvam o dia

```
// não quebra se o caminho não existir  
usuario?.endereco?.cidade  
  
// padrão só se for null/undefined (0 continua 0)  
config.limite ?? 10  
  
// o idioma entre front e back  
JSON.stringify(obj)    // objeto → texto  
JSON.parse(texto)      // texto → objeto
```

?? × || — a diferença que importa

```
0 || 10 // 10 🤖 zero é "falso"
0 ?? 10 // 0 ✅
```

```
"" || "padrão" // "padrão"
"" ?? "padrão" // ""
```

|| troca quando é **falso** (0, "", false, null)

?? troca só quando **não existe** (null, undefined)

Para valor padrão, prefira ??

? . — não quebra o programa

```
usuario.endereco.cidade // ✨ erro se endereco não existir
```

```
usuario?.endereco?.cidade // undefined, sem quebrar
```

```
corpo.erros?.join(", ") // chama join SÓ se erros existir
```

Leia como: "se existir, continua; senão, para aqui".

Recapitulando

1. `map` transforma · `filter` seleciona · `reduce` resume
2. Desestruturação tira de dentro · `...` copia e espalha
3. **Copie, mude a cópia, devolva a cópia**

Mão na massa 🚀

`aula-02/exercicios/README.md`

Próxima: o que acontece quando o código precisa **esperar**